

Sintaxis y Semántica de Lenguajes

Guía de Estudio — Tema 02

Matías Gel

Ciclo lectivo 2026

Índice de Contenidos

1. Guía de Estudio — Tema 02: Sintaxis y Semántica de Lenguajes	1
1.1. 1. Introducción al tema	2
1.2. 2. Objetivos de aprendizaje	2
1.3. 3. Conceptos previos necesarios	3
1.4. 4. Desarrollo teórico	3
1.4.1. 4.1 Sintaxis y semántica: forma vs. significado	3
1.4.2. 4.2 Estructura léxica: del texto a los tokens	4
1.4.3. 4.3 Gramáticas formales: BNF y EBNF	6
1.4.4. 4.4 EBNF: notación extendida para gramáticas más legibles	11
1.4.5. 4.5 Diagramas de sintaxis (<i>railroad diagrams</i>)	13
1.4.6. 4.6 Semántica: estática y dinámica	13
1.4.7. 4.7 Pipeline compilador/intérprete y vínculo con IA	14
1.5. 5. Ejemplos trabajados (paso a paso)	15
1.5.1. Ejemplo 1: Identificar errores en TypeScript	15
1.5.2. Ejemplo 2: Derivación completa de $A := B * (A + C)$	16
1.5.3. Ejemplo 3: Ejercicio del material de cátedra — derivar $C := D - E * F$	16
1.5.4. Ejemplo 4: BNF a EBNF — sentencia de asignación con expresión entera	17
1.6. 6. Puntos clave y resumen	17
1.7. 7. Autoevaluación (distintas al TP)	18
1.8. 8. Glosario	19
1.9. 9. Referencias y lecturas recomendadas	20
1.9.1. Bibliografía obligatoria	20
1.9.2. Lecturas recomendadas para profundizar	20
1.10. 10. Límites de alcance de esta guía	20

1. Guía de Estudio — Tema 02: Sintaxis y Semántica de Lenguajes

Materia: Paradigmas y Lenguajes de Programación 2026

Institución: Universidad Nacional de Tierra del Fuego — Instituto IDEI

Tema: 02 — Sintaxis y Semántica de Lenguajes

Semana / Clase: Semana 1, Clase 2 de 2

Docente: Matías Gel

Fecha: 2026-03-20

1.1. 1. Introducción al tema

En este tema vas a responder dos preguntas centrales:

1. ¿Cómo decide un compilador si un programa está bien escrito?
2. ¿Qué diferencia hay entre que un programa esté bien formado y que tenga sentido?

La idea clave es que todo lenguaje de programación se define por dos capas:

“Un lenguaje de programación es una notación formal para describir algoritmos a ser ejecutados por computadoras.”

— Cátedra Paradigmas y Lenguajes de Programación, UNTDF

- **Sintaxis:** conjunto de reglas que determinan cuándo un programa está bien formado, considerando solo el punto de vista de la representación.
- **Semántica:** le asigna significado a los programas sintácticamente correctos.

El objetivo principal de esta guía es que domines las herramientas formales de descripción sintáctica: **BNF** y **EBNF**. Las secciones sobre léxico, semántica y pipeline son contexto secundario igualmente importante para entender el cuadro completo.

1.2. 2. Objetivos de aprendizaje

Al terminar esta guía deberías poder:

1. Diferenciar errores sintácticos, semánticos estáticos y semánticos dinámicos con ejemplos concretos.
 2. Explicar qué hacen el lexer y el parser y por qué están separados.
 3. **Leer, escribir e interpretar reglas BNF y EBNF.** ← objetivo central
 4. Construir derivaciones completas paso a paso desde un símbolo inicial.
 5. Dibujar e interpretar árboles sintácticos a partir de una derivación.
 6. Detectar si una gramática es ambigua y explicar por qué es un problema de diseño.
 7. Convertir reglas BNF en reglas EBNF y viceversa.
 8. Leer diagramas de sintaxis (railroad diagrams) como especificación visual equivalente.
 9. Entender, a nivel panorámico, la diferencia entre semántica estática y dinámica.
-

1.3. 3. Conceptos previos necesarios

Para estudiar este tema con comodidad, necesitás recordar:

- Qué es un compilador y qué diferencia hay con un intérprete (Tema 01).
- Nociones básicas de expresiones, tipos y sentencias en TypeScript.
- Idea intuitiva de lo que es una regla gramatical.

Si te falta alguno de estos puntos, repasá primero las filminas del Tema 01.

1.4. 4. Desarrollo teórico

1.4.1. 4.1 Sintaxis y semántica: forma vs. significado

Ver filminas: F-02, F-03, F-04, F-05, F-06, F-06b.

1.4.1.1. 4.1.1 Definición formal La definición de un lenguaje permite determinar:

- Si un programa es válido.
- Cuál es su significado o efecto.

Sintaxis es el conjunto de reglas y criterios de escritura que permiten la formación de programas correctos en un lenguaje. Las reglas se dividen en dos tipos:

- **Reglas léxicas:** definen el conjunto de caracteres que constituyen el alfabeto del lenguaje y la forma de combinar dichos caracteres para formar palabras válidas. Por ejemplo, Java y Python consideran en forma diferente las letras mayúsculas y minúsculas.
- **Reglas sintácticas:** definen cómo pueden formarse las sentencias a partir de constituyentes básicos llamados palabras.

Semántica es el significado de expresiones, sentencias y unidades de programa.

Ejemplo canónico:

- **Sintaxis** de la sentencia condicional en C: `if (<expresión>) <sentencia>`
- **Semántica** de esa sentencia: *“Si el valor actual de la expresión es cierto, se ejecuta la sentencia siguiente.”*

1.4.1.2. 4.1.2 Criterios de una buena sintaxis

Criterio	Qué garantiza
Legibilidad	El código escrito puede leerse y entenderse
Facilidad de escritura	Es natural expresar algoritmos
Facilidad de verificación	Es posible demostrar propiedades del programa

Criterio	Qué garantiza
Facilidad de traducción	El compilador puede procesarlo eficientemente
Carencia de ambigüedad	Cada construcción tiene exactamente un significado

1.4.1.3. 4.1.3 Tres categorías de error en TypeScript

Error	Tipo	Detectado por
<code>const x: number = "hola"</code>	Semántico estático	Type checker (compilación)
<code>if true { console.log("ok") }</code>	Sintáctico	Parser (compilación)
<code>const y = undefined; y.length</code>	Semántico dinámico	Runtime (ejecución)

Esta tabla es clave: conviene memorizarla y poder reproducirla con ejemplos propios.

1.4.2. 4.2 Estructura léxica: del texto a los tokens

Ver filmas: F-07 a F-11.

Antes de que el parser pueda analizar la estructura gramatical de un programa, el texto fuente debe transformarse en unidades mínimas. Ese trabajo lo hace el **lexer (analizador léxico)**.

1.4.2.1. 4.2.1 Elementos sintácticos de un LP Todo lenguaje define cómo se construyen las siguientes categorías:

- **Caracteres:** conjunto de símbolos permitidos (ASCII, Unicode).
- **Identificadores:** nombres para variables, funciones, tipos. Reglas típicas: deben comenzar con letra, pueden incluir dígitos, algunos lenguajes son case-sensitive (Java, TypeScript) y otros no.
- **Operadores:** aritméticos (+, -, *, /), lógicos (&&, ||), relacionales (<, >=), de asignación (=, :=).
- **Palabras clave y reservadas:** `if`, `while`, `const`, `function` — no pueden usarse como identificadores.
- **Comentarios:** texto ignorado por el compilador (`//`, `/* */`).
- **Espacios en blanco:** generalmente ignorados, salvo en lenguajes donde la indentación es significativa (Python).

- **Delimitadores y corchetes:** {}, (), [], ; — estructuran el programa.
- **Expresiones:** combinaciones de operandos y operadores que producen un valor.
- **Sentencias:** unidades de ejecución del programa.

1.4.2.2. 4.2.2 Lexemas y tokens Dos conceptos fundamentales:

- **Lexema:** unidad sintáctica de más bajo nivel. Incluye identificadores, operadores y palabras especiales. Es la instancia concreta.
- **Token:** categoría del lexema. Es la abstracción.

Ejemplo clásico para la sentencia `indice = 5 * contador + 1;;`

Lexema	Token
<code>indice</code>	identificador
<code>=</code>	op-asignación
<code>5</code>	constante_entera
<code>*</code>	op_producto
<code>contador</code>	identificador
<code>+</code>	op_suma
<code>1</code>	constante_entera
<code>;</code>	delimitador

Otro ejemplo (Sebesta, Cap. 4) para `result = oldsum - value / 100;;`

Lexema	Token
<code>result</code>	IDENT
<code>=</code>	ASSIGN_OP
<code>oldsum</code>	IDENT
<code>-</code>	SUB_OP
<code>value</code>	IDENT
<code>/</code>	DIV_OP
<code>100</code>	INT_LIT
<code>;</code>	SEMICOLON

1.4.2.3. 4.2.3 El analizador léxico El lexer es esencialmente un **reconocedor de patrones**.

Su función es:

1. Tomar la cadena de caracteres del programa fuente.
2. Agrupar e identificar lexemas.

3. Descartar espacios en blanco y comentarios.
4. Clasificar lexemas en tokens.
5. Devolver tokens de a uno al parser.

Sebesta explica por qué el análisis léxico se separa del sintáctico con tres razones (Cap. 4):

1. **Simplicidad:** las técnicas de análisis léxico son menos complejas que las sintácticas.
 2. **Eficiencia:** permite optimizar el lexer por separado, ya que consume una porción significativa del tiempo de compilación.
 3. **Portabilidad:** el lexer es la parte dependiente de plataforma (lectura de archivos, bufferización); el parser puede ser independiente.
-

1.4.3. 4.3 Gramáticas formales: BNF y EBNF

Ver filmas: F-12 a F-20.

Esta es la sección más importante de la guía. Tomá tiempo para trabajar cada ejemplo de forma activa: cubrí la derivación, intentá hacerla vos, y después compará.

1.4.3.1. 4.3.1 Por qué se necesitan gramáticas formales Describir la sintaxis de un lenguaje en lenguaje natural (castellano, inglés) produce ambigüedades y omisiones.

FORTTRAN fue originalmente definido con reglas en inglés.

Python tiene su gramática oficial publicada como BNF en docs.python.org.

— Cátedra Paradigmas y Lenguajes de Programación, UNTDF

Usando BNF, como señala Sebesta (Cap. 4), se obtienen al menos tres ventajas:

1. Las descripciones BNF son **claras y concisas**, tanto para humanos como para herramientas de software.
2. La descripción BNF puede usarse como **base directa para el analizador sintáctico**.
3. Las implementaciones basadas en BNF son **fáciles de mantener** por su modularidad.

1.4.3.2. 4.3.2 Gramáticas libres de contexto — clasificación de Chomsky En 1959, Chomsky propuso una clasificación de las gramáticas formales. Los lenguajes de programación utilizan **gramáticas libres de contexto**: un conjunto de reglas que definen todas las construcciones válidas de un lenguaje.

Formalmente, una gramática se define como la tupla (N, T, S, P) :

Componente	Significado	Ejemplo
N	Conjunto de símbolos no terminales (abstracciones)	<code><assign></code> , <code><expr></code> , <code><id></code>
T	Conjunto de símbolos terminales (tokens concretos)	A, B, <code>:=</code> , <code>+</code> , <code>(</code> , <code>)</code>
S	Símbolo inicial (la meta a derivar)	<code><assign></code>
P	Conjunto de reglas de producción	<code><assign> ::= <id> := <expr></code>

Símbolos no terminales son abstracciones: representan categorías sintácticas que se deben expandir con más reglas.

Símbolos terminales son los lexemas reales del programa: no se expanden más.

1.4.3.3. 4.3.3 El metalenguaje BNF: metasímbolos y estructura de reglas BNF (Backus-Naur Form) es el metalenguaje estándar para escribir gramáticas formales. Sus metasímbolos son:

Símbolo	Significado
<code>::=</code>	“se define como”
<code>.</code>	fin de una definición
<code>\ </code>	“or” lógico — alternativa
<code>< ></code>	encierran nombres de no terminales
<code>*</code>	cero o más ocurrencias del elemento precedente
<code>+</code>	una o más ocurrencias del elemento previo

Los terminales se escriben tal y como son (sin `< >`).

Estructura de una regla BNF:

```
<LHS> ::= <RHS>
```

- **LHS** (*Left-Hand Side*): el símbolo no terminal que se está definiendo.
- **RHS** (*Right-Hand Side*): la definición, que puede combinar terminales y no terminales.

Ejemplo de regla simple:

```
<assign> ::= <var> = <expression>
```

Se lee: “La abstracción `<assign>` se define como una instancia de `<var>`, seguida del lexema `=`, seguida de una instancia de `<expression>`.”

Ejemplo de la sentencia `if` en Pascal con alternativas:

```

<enunc_if> ::= if <expr_log> then <enunc>
           | if <expr_log> then <enunc> else <enunc>

```

Ambas reglas para <enunc_if> pueden escribirse en una sola gracias al símbolo |.

Se lee: “<enunc_if> puede ser un *if-then* o un *if-then-else*.”

1.4.3.4. 4.3.4 Gramática de trabajo para los ejercicios Vamos a trabajar con la siguiente gramática a lo largo de toda esta sección. Es el ejemplo canónico de la cátedra:

```

<assign> ::= <id> := <expr>
<id>      ::= A | B | C
<expr>    ::= <id> + <expr>
           | <id> * <expr>
           | (<expr>)
           | <id>

```

Descripción informal:

- Hay una única sentencia de asignación (:=).
- Los únicos identificadores válidos son A, B y C.
- Las expresiones pueden combinar sumas, productos y paréntesis.

Esta gramática describe un mini-lenguaje. El objetivo no es realismo sino que derivar sea manejable.

1.4.3.5. 4.3.5 Derivaciones: del símbolo inicial a la cadena final Una **derivación** es el proceso de generar una cadena aplicando reglas de la gramática, empezando por el símbolo inicial y reemplazando no terminales hasta que toda la cadena son terminales.

Notación:

- El símbolo $\Rightarrow$ se lee “**deriva**”.
- Cada línea reemplaza **exactamente un** no terminal por una de sus definiciones.
- Cada string intermedio (incluyendo el inicial <assign>) se llama **forma sentencial** (*sentential form*).

Derivación completa de A := B * (A + C):

<assign>	
<id> := <expr>	[aplicar: <assign> ::= <id> := <expr>]
A := <expr>	[aplicar: <id> ::= A]
A := <id> * <expr>	[aplicar: <expr> ::= <id> * <expr>]
A := B * <expr>	[aplicar: <id> ::= B]
A := B * (<expr>)	[aplicar: <expr> ::= (<expr>)]

$A := B * (<id> + <expr>)$	[aplicar: $<expr> ::= <id> + <expr>$]
$A := B * (A + <expr>)$	[aplicar: $<id> ::= A$]
$A := B * (A + <id>)$	[aplicar: $<expr> ::= <id>$]
$A := B * (A + C)$	[aplicar: $<id> ::= C$]

Todos los símbolos son ahora terminales: **la derivación terminó.**

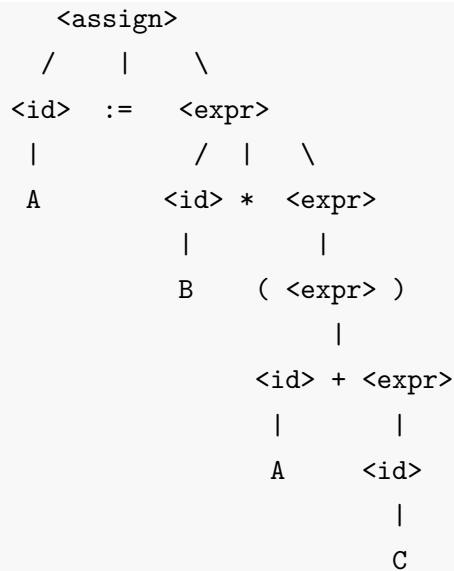
Estrategia de estudio: antes de leer la solución, intentá derivar $A := B * (A + C)$ solo con la gramática y el símbolo inicial. Verificá que cada corresponde a una producción válida.

1.4.3.6. 4.3.6 Árboles sintácticos (*parse trees*) Un **árbol sintáctico** representa la estructura jerárquica de la derivación. Es la misma información que la secuencia de , pero organizada de forma visual.

Reglas del árbol:

- Los **nodos internos** están etiquetados con no terminales.
- Los **nodos hoja** están etiquetados con terminales.
- Los **hijos de cada nodo** representan el reemplazo del no terminal en un paso de la derivación.

Árbol para $A := B * (A + C)$:



Para estudiar: dibujá el árbol de forma independiente a partir de la derivación del punto anterior. El árbol y la derivación deben ser consistentes entre sí.

1.4.3.7. 4.3.7 Gramáticas ambiguas

Una gramática es ambigua si permite construir dos o más árboles de derivación distintos para la misma cadena.

Esto implica que hay dos significados posibles para el mismo texto. En un compilador, la ambigüedad es un **defecto de diseño**: el compilador no puede elegir de forma única el árbol correcto.

Ejemplo con $J := 1 + 2 * 3$:

Supongamos esta gramática simplificada para expresiones:

```
<expr> ::= <num> + <expr>
        | <num> * <expr>
        | <num>
<num>  ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Esta gramática genera dos árboles distintos para $1 + 2 * 3$:

Árbol A (agrupa $1 + 2$ primero $\rightarrow$ resultado es $3 * 3 = 9$):

```

      <expr>
     /  |  \
  <num> + <expr>
   |       / | \
   1      <num> * <expr>
           |       |
           2      <num>
                   |
                   3
```

Árbol B (agrupa $2 * 3$ primero $\rightarrow$ resultado es $1 + 6 = 7$):

```

      <expr>
     /  |  \
  <num> * <expr>
           / | \
           <expr> + <num>
          / | \   |
  <num> + <expr>  3
   |       |
   1      <num>
           |
           2
```

Dos árboles, dos valores distintos. **La gramática es ambigua.**

Solución: rediseñar la gramática para imponer **precedencia** (multiplicación antes que suma) y **asociatividad** (izquierda o derecha). Esto se hace estratificando los no terminales:

```
<expr>    ::= <expr> + <term> | <term>
<term>    ::= <term> * <factor> | <factor>
<factor>  ::= <num> | (<expr>)
<num>     ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Ahora $*$ tiene mayor precedencia porque está más profundo en la jerarquía: primero se resuelven los $\langle \text{factor} \rangle$, luego los $\langle \text{term} \rangle$ y finalmente las $\langle \text{expr} \rangle$.

1.4.4. 4.4 EBNF: notación extendida para gramáticas más legibles

Ver filminas: F-19, F-20.

EBNF (Extended BNF) agrega tres abreviaciones sobre BNF, sin aumentar la potencia formal:

Símbolo	Significado	Equivalencia en BNF
$[x]$	x es opcional (0 o 1 vez)	$\langle A \rangle ::= x \mid$
$\{x\}$ o $\{x^*\}$	x se repite 0 o más veces	$\langle A \rangle ::= \langle A \rangle x \mid$
$\mid$ dentro de $[]$	alternativa en contexto de opcionalidad	

1.4.4.1. Ejemplo: valor con signo opcional

En BNF puro necesitarías:

```
<valor>    ::= <valor_pos> | <valor_neg> | <num_sin_signo>
<valor_pos> ::= + <num_sin_signo>
<valor_neg> ::= - <num_sin_signo>
```

En EBNF se simplifica:

```
<valor> ::= [<signo>] <num_sin_signo>
<signo> ::= + | -
```

Mucho más legible. El $[\langle \text{signo} \rangle]$ dice: el signo puede estar o no.

1.4.4.2. 4.4.1 Gramática EBNF completa para un LP simple Este es el ejemplo más completo del material de cátedra. Leelo con cuidado porque aparece tanto en las filminas como en este ejercicio:

Reglas sintácticas:

```

<programa>      ::= { <sentencia>* }
<sentencia>     ::= <asignación> | <condicional> | <loop>
<asignación>    ::= <identificador> = <expr>
<condicional>   ::= if <expr> { <sentencia>* }
                  | if <expr> { <sentencia>* } else { <sentencia>* }
<loop>          ::= while <expr> { <sentencia>* }
<expr>          ::= <identificador> | <número> | (<expr>) | <expr> <operador> <expr>

```

Reglas léxicas:

```

<operador>      ::= + | - | * | / | = | | < | > | |
<identificador> ::= <letra> <id>*
<id>            ::= <letra> | <dígito>
<número>        ::= <dígito>+
<letra>         ::= a | b | c | ... | z
<dígito>        ::= 0 | 1 | 2 | ... | 9

```

Puntos importantes de esta gramática:

- { <sentencia>* } significa cero o más sentencias entre llaves.
- El condicional tiene forma if-then (sin else) y if-then-else. Las dos variantes se separan con |.
- El <loop> es simplemente un while.
- Las **reglas léxicas** definen los tokens (identificadores, números, operadores) usando las mismas herramientas BNF/EBNF.

1.4.4.3. 4.4.2 Comparación BNF vs. EBNF para el mismo fragmento Sentencia de asignación simple:

BNF:

```

<asig>          ::= <id> = <expr>
<expr>          ::= <expr> + <term> | <term>
<term>          ::= <term> * <factor> | <factor>
<factor>        ::= <id> | <num> | (<expr>)

```

EBNF equivalente:

```

<asig>          ::= <id> = <expr>
<expr>          ::= <term> { (+|-) <term> }
<term>          ::= <factor> { (*|/) <factor> }
<factor>        ::= <id> | <num> | (<expr>)

```

Ambas describen lo mismo; EBNF evita la recursión izquierda en las reglas de `<expr>` y `<term>` y resulta más legible.

1.4.5. 4.5 Diagramas de sintaxis (*railroad diagrams*)

Ver filminas: F-21, F-22, F-23.

Los diagramas de sintaxis son representaciones **gráficas** equivalentes a las reglas EBNF. Se usan mucho en documentación oficial de lenguajes.

Convención visual:

- Un **rectángulo** representa un no terminal.
- Un **óvalo** o **círculo** representa un terminal.
- Una cadena es válida si puede “**recorrer**” el diagrama de izquierda a derecha.

Ejemplo para el condicional de la gramática anterior:

```
(if)  [expr]  ({)  {[sent]*}  (})
                                     (else)  ({)  {[sent]*}  (})
```

Lectura:

- Camino superior: `if <expr> { ... } (sin else)`.
- Camino inferior: `if <expr> { ... } else { ... }`.

Dos usos principales:

1. Ayuda al programador a saber cómo escribir un programa sintácticamente correcto.
2. Se utiliza para determinar cuándo un programa es sintácticamente correcto (lo que hace el compilador).

1.4.6. 4.6 Semántica: estática y dinámica

Ver filminas: F-24 a F-28.

Una vez que el parser construye el árbol sintáctico, el compilador necesita verificar que el programa también tiene sentido. Aquí entra la semántica.

1.4.6.1. 4.6.1 Semántica estática Se verifica **antes de ejecutar** el programa.

El problema es que no toda restricción semántica puede expresarse con BNF. Por ejemplo, la compatibilidad de tipos en una asignación depende del contexto (qué tipo declaró la variable): eso es sensible al contexto, no libre de contexto.

Solución: **gramáticas de atributos** (Knuth, 1968). Extienden BNF agregando atributos (como el tipo) a los símbolos y reglas que calculan esos atributos sobre el árbol.

Ejemplo con TypeScript:

```
const x: number = "texto"
// → Type 'string' is not assignable to type 'number'
```

- El **parser** construyó el árbol sin error (la forma es válida).
- El **type checker** evalúa atributos de tipo sobre ese árbol y rechaza.

Esto es semántica estática en acción.

1.4.6.2. 4.6.2 Semántica dinámica Describe el comportamiento **en tiempo de ejecución**.

Existen tres enfoques formales de descripción (Sebesta, Cap. 3):

Enfoque	Idea central	Referencia
Operacional	El significado = secuencia de pasos en una máquina abstracta	Landin (1964); Plotkin (1981)
Denotacional	El significado = función matemática estado → estado	Scott & Strachey (1970)
Axiomática	El significado = aserciones lógicas (pre/postcondiciones)	Hoare (1969)

En este tema vemos solo la panorámica. El tratamiento formal completo queda fuera del scope.

1.4.7. 4.7 Pipeline compilador/intérprete y vínculo con IA

Ver filmintas: F-29 a F-34.

Todo lo visto en este tema se integra en el pipeline de compilación:

Código fuente

```
↓ Lexer      → Tokens
↓ Parser     → Árbol sintáctico
↓ Type checker → Árbol anotado (semántica estática)
```

↓ Generación	→ Código target (compilador)
ó Ejecución	→ Resultado (intérprete)

Vínculo con IA generativa (*constrained decoding*):

Las gramáticas formales vuelven a ser infraestructura activa. Forzar que la salida de un modelo siga una gramática EBNF garantiza que el JSON, código o estructura generada sea siempre válida.

Línea del tiempo:

1957 — Chomsky: gramáticas formales.

1960 — Backus-Naur: BNF para Algol60.

2023 — Constrained decoding en LLMs (Willard & Louf).

2026 — Structured Prompt Language con EBNF (Gong, arXiv:2602.21257).

1.5. 5. Ejemplos trabajados (paso a paso)

1.5.1. Ejemplo 1: Identificar errores en TypeScript

```
// Línea 1:
const x: number = "hola"

// Línea 2:
if true { console.log("ok") }

// Línea 3:
const y = undefined; y.length
```

Análisis:

Línea 1: - La forma sintáctica es válida (operador de asignación, literales, tipos genéricos). - El valor "hola" es de tipo `string`, pero la variable fue declarada como `number`. - El **type checker** lo rechaza en compilación. - Categoría: **error semántico estático**.

Línea 2: - La regla sintáctica del `if` en TypeScript exige `if (<expr>) ...` — con paréntesis. - Falta el paréntesis que envuelve `true`. - El **parser** no puede construir el árbol. - Categoría: **error sintáctico**.

Línea 3: - La forma sintáctica es válida. - y está declarado, así que pasa el type checker básico (en configuraciones estrictas de TypeScript puede detectarlo). - En **ejecución**, acceder a `.length` de `undefined` lanza `TypeError`. - Categoría: **error semántico dinámico**.

1.5.2. Ejemplo 2: Derivación completa de $A := B * (A + C)$

Gramática:

```

<assign> ::= <id> := <expr>
<id>      ::= A | B | C
<expr>    ::= <id> + <expr> | <id> * <expr> | (<expr>) | <id>

```

Derivación paso a paso:

```

<assign>
  <id> := <expr>                (producción 1: expandir <assign>)
  A := <expr>                   (producción 2: <id> → A)
  A := <id> * <expr>             (producción 3: <expr> → <id> * <expr>)
  A := B * <expr>               (producción 2: <id> → B)
  A := B * (<expr>)              (producción 4: <expr> → (<expr>))
  A := B * (<id> + <expr>)        (producción 3: <expr> → <id> + <expr>)
  A := B * (A + <expr>)          (producción 2: <id> → A)
  A := B * (A + <id>)            (producción 5: <expr> → <id>)
  A := B * (A + C)              (producción 2: <id> → C)

```

Verificación: todos los símbolos son terminales. La derivación está completa.**Árbol resultante:**

```

      <assign>
      /  |  \
    <id> := <expr>
    |      / | \
    A      <id> * <expr>
           |      |
           B      (<expr>)
                |
                <id> + <expr>
                |      |
                A      <id>
                     |
                     C

```

1.5.3. Ejemplo 3: Ejercicio del material de cátedra — derivar $C := D - E * F$

Gramática extendida con el identificador D, E, F y el operador -:


```

<assign> ::= <id> := <expr>
<id>      ::= A | B | C | D | E | F
<expr>    ::= <id> + <expr> | <id> * <expr> | <id> - <expr>
           | (<expr>) | <id>

```

Una derivación posible:

```

<assign>
  <id> := <expr>
  C := <expr>
  C := <id> - <expr>
  C := D - <expr>
  C := D - <id> * <expr>
  C := D - E * <expr>
  C := D - E * <id>
  C := D - E * F

```

Pregunta para pensar: ¿existe otra derivación que produzca $C := D - E * F$? Si la hay, ¿qué implica eso sobre la gramática? (Revisar sección 4.3.7.)

1.5.4. Ejemplo 4: BNF a EBNF — sentencia de asignación con expresión entera

En BNF, una expresión aritmética con suma y producto puede escribirse así:

```

<expr>    ::= <expr> + <term> | <term>
<term>    ::= <term> * <factor> | <factor>
<factor>  ::= <id> | ( <expr> )

```

En EBNF equivalente:

```

<expr>    ::= <term> { + <term> }
<term>    ::= <factor> { * <factor> }
<factor>  ::= <id> | ( <expr> )

```

Diferencias clave: - La EBNF usa { } para iterar, eliminando las recursiones explícitas. - Ambas gramáticas generan el mismo conjunto de cadenas. - La EBNF suele usarse más en documentación por ser legible; BNF sirve de base para parsers.

1.6. 6. Puntos clave y resumen

Bloque principal: Gramáticas formales

- Una gramática libre de contexto es la tupla (N, T, S, P) .
- En BNF, cada regla tiene un LHS (no terminal) y un RHS (definición con terminales y no terminales).
- Una **derivación** aplica reglas desde S hasta obtener una cadena de terminales.
- Un **árbol sintáctico** organiza la misma información de forma jerárquica.
- Una gramática es **ambigua** si la misma cadena tiene dos árboles posibles $\rightarrow$ dos significados $\rightarrow$ defecto de diseño.
- EBNF extiende BNF con $[x]$ (opcional) y $\{x\}$ (repetición) sin aumentar la potencia formal.
- Los **diagramas de sintaxis** son la representación gráfica de las mismas reglas.

Bloque secundario: contexto completo

- Sintaxis = forma; semántica = significado.
- Lexer convierte texto en tokens; parser convierte tokens en árbol.
- La separación lexer/parser es por simplicidad, eficiencia y portabilidad.
- Semántica estática = verificación antes de ejecutar (tipos, etc.); dinámica = comportamiento en ejecución.
- El pipeline es: Lexer $\rightarrow$ Parser $\rightarrow$ Type checker $\rightarrow$ Generación/Ejecución.

Mapa conceptual:

Texto fuente

```

Lexer           $\rightarrow$  secuencia de tokens
Parser          $\rightarrow$  árbol sintáctico (usando BNF/EBNF)
Type checker    $\rightarrow$  árbol anotado (semántica estática)
Código/Ejecución (semántica dinámica)

```

1.7. 7. Autoevaluación (distintas al TP)

Preguntas conceptuales:

1. Definí con tus palabras la diferencia entre terminal y no terminal. Da un ejemplo de cada uno usando la gramática de trabajo de la sección 4.3.4.
2. ¿Qué es una forma sentencial? ¿En qué se diferencia de la cadena final de una derivación?
3. Explicá por qué una gramática ambigua es un defecto de diseño del lenguaje, con un ejemplo.
4. ¿Qué ventajas tiene EBNF sobre BNF puro? ¿Implica mayor poder expresivo?

Ejercicios de derivación:

5. Usando la gramática de la sección 4.3.4, derivá la cadena $B := A + C$. Mostrá cada paso con la producción aplicada.

6. Usando la misma gramática, dibujá el árbol sintáctico de $A := (B + C) * A$.
7. Determiná si la siguiente gramática es ambigua para la cadena $1 + 2 + 3$. Justificá:

```
<expr> ::= <expr> + <expr> | <num>
<num>  ::= 1 | 2 | 3
```

Preguntas de aplicación:

8. ¿Qué herramienta del pipeline de compilación (lexer, parser, type checker, runtime) detecta cada uno de los siguientes?
 - Un paréntesis que no cierra.
 - Una variable usada con tipo incorrecto.
 - Un acceso a índice fuera de rango en un arreglo.
9. Escribí en EBNF una regla para una lista de identificadores separados por comas (al menos uno).
10. Dado un diagrama de sintaxis para un `while`, describí en una frase qué cadenas acepta.

Sugerencias para verificar tus respuestas:

- En los ejercicios de derivación: verificá que cada `use` exactamente una producción de la gramática y que el resultado final no tenga `< >`.
- Si dudás entre dos derivaciones posibles para el mismo resultado, la gramática puede ser ambigua.

1.8. 8. Glosario

- **Token:** categoría léxica de un lexema (por ejemplo, `identificador`, `op_suma`).
- **Lexema:** secuencia concreta de caracteres reconocida por el lexer (`contador`, `+`, `42`).
- **Lexer:** componente que transforma texto fuente en tokens, descartando espacios y comentarios.
- **Parser:** componente que verifica estructura sintáctica y construye el árbol.
- **BNF** (*Backus-Naur Form*): notación formal para especificar gramáticas libres de contexto con reglas $LHS ::= RHS$.
- **EBNF** (*Extended BNF*): versión extendida de BNF con `[ ]` (opcional) y `{ }` (repetición).
- **Gramática libre de contexto:** gramática de la forma (N, T, S, P) donde las producciones reemplazan un no terminal con independencia del contexto.
- **No terminal:** símbolo abstracto que se expande mediante reglas de gramática (`<expr>`, `<id>`).
- **Terminal:** símbolo concreto final de la gramática, equivalente a un token (`A`, `:=`, `+`).
- **Símbolo inicial (S):** no terminal desde donde comienza toda derivación.
- **Derivación:** secuencia de aplicaciones de reglas desde `S` hasta una cadena de terminales.

- **Forma sentencial:** cualquier string intermedio durante una derivación (incluyendo el inicial).
 - **Árbol sintáctico** (*parse tree*): representación jerárquica de la derivación; nodos internos = no terminales, hojas = terminales.
 - **Ambigüedad gramatical:** existencia de más de un árbol de derivación válido para la misma cadena.
 - **Diagrama de sintaxis** (*railroad diagram*): representación gráfica de reglas EBNF; una cadena es válida si puede recorrerse el diagrama de izquierda a derecha.
 - **Semántica estática:** reglas verificables antes de ejecutar (tipos, declaraciones antes del uso, etc.).
 - **Semántica dinámica:** comportamiento efectivo del programa durante la ejecución.
 - **Type checker:** componente que verifica restricciones semánticas estáticas (tipos, compatibilidad).
 - **Constrained decoding:** técnica que usa una gramática formal para guiar la generación de texto en modelos de lenguaje.
-

1.9. 9. Referencias y lecturas recomendadas

1.9.1. Bibliografía obligatoria

- **Material de cátedra UNTDF 2025:** Slides «Sintaxis de Lenguajes de Programación». Fuente principal para BNF, EBNF, derivaciones, árboles y diagramas de sintaxis.
- **Sebesta, R.W.** (2019). *Concepts of Programming Languages*, 12^a ed., Addison Wesley. Capítulo 3 (gramáticas formales, BNF, ambigüedad, semántica estática y dinámica) y Capítulo 4 (análisis léxico y sintáctico).
- **Gabbrielli, M. & Martini, S.** (2023). *Programming Languages: Principles and Paradigms*. Capítulo 4 (nombres, entorno y binding). Puente hacia el Tema 09.

1.9.2. Lecturas recomendadas para profundizar

- Sebesta Cap. 3 completo: lectura propia de la fuente original sobre BNF, EBNF en detalle.
 - **Willard & Louf** (2023). *Efficient Guided Generation for Large Language Models*. arXiv:2307.09702 — constrained decoding.
 - **Gong, L.** (2026). *Structured Prompt Language*. arXiv:2602.21257 — gramáticas en prompts.
-

1.10. 10. Límites de alcance de esta guía

Para mantener coherencia con el alcance definido para el tema:

- No se desarrolla la implementación de parsers LL/LR ni la construcción algorítmica de analizadores.
- No se profundizan los formalismos de semántica operacional, denotacional ni axiomática más allá de la síntesis conceptual.
- El tratamiento exhaustivo de nombres, binding, entorno y scope rules queda para el Tema 09.

Esta guía está pensada para que puedas estudiar de forma completamente autónoma sin salirte del scope definido para el Tema 02.